Application Development Code Review and Suggestion Agent

PROBLEM STATEMENT

Development teams require efficient code review processes to maintain code quality and accelerate delivery. Manual reviews are time-consuming and vary in consistency, especially under tight deadlines. Automated static analysis tools provide limited context-aware feedback and do not integrate well with development workflows. An AI-powered multi-agent system that analyzes code changes, provides detailed review comments, and integrates with code repositories and CI/CD pipelines can streamline code reviews and improve quality.

Data Considerations:

Source code and pull request metadata from Git repositories; build and test results from CI/CD systems; historical code review comments; moderate data volume with frequent updates; preprocessing for syntax parsing and change detection; privacy adherence for proprietary code; synthetic pull requests with known issues for training.

Consider using synthetic or anonymized data where appropriate. Ensure data quality and relevance to the problem context.

SOLUTION EXPECTATIONS

A multi-agent AI system with agents for code analysis, issue detection (style, bugs, security), and suggestion generation. Features include a web UI integrated with repository platforms showing inline comments and summary reports. Integrations with CI/CD pipelines enable automated review triggers and feedback loops. Success metrics include review relevance, reduction in manual review time, and defect detection rate. Demonstrations cover pull request analysis, multi-agent workflows, and testing with comprehensive documentation.

Feel free to explore creative solutions that align with the problem context and objectives.

Use the following checklist as applicable while solving the selected problem statement.

USER EXPERIENCE AND INTERFACE

- Design an intuitive interface aligned to the target workflow, user journey, context, tone, and accessibility needs.
- Use conversation, visualization, or a hybrid experience where it improves clarity, personalization, and speed of action.

DATA ARCHITECTURE AND PROCESSING

- Use relevant, high-quality, anonymized or synthetic data with clear preprocessing, retrieval, aggregation, and correlation methods.
- Choose suitable storage for structured and unstructured data, ensuring consistency across sources, generated datasets, and downstream outputs.

CORE AI SOLUTION DESIGN AND AGENTIC ARCHITECTURE

- Define the agentic design pattern, agent roles, collaboration model, handoffs, tool usage, retry, reflection, and escalation logic.
- Balance autonomy with human approval checkpoints, intervention points, and clear boundaries for high-risk or uncertain decisions.
- Ground outputs in trusted sources, cite evidence where useful, separate facts from assumptions, and explain reasoning, confidence, limits, and trade-offs.
- Build responsible AI guardrails for privacy, bias, explainability, prompt injection, unsafe recommendations, unauthorized tool execution, and regulated decisions.

TECHNICAL IMPLEMENTATION

- Use a modular, scalable architecture that clearly separates AI layers, enterprise systems, knowledge stores, APIs, tools, and external feeds.
- Define context engineering across memory, caching, retrieval, prompts, context windows, token usage, latency, cost, and privacy.
- Maintain configurability across models, prompts, tools, APIs, vector stores, and deployment environments to avoid lock-in.
- Cover security, identity, permissions, secrets, data retention, audit logging, access governance, and misuse monitoring.

TESTING AND QUALITY ASSURANCE

- Evaluate beyond demo success: accuracy, groundedness, hallucination control, latency, cost, reliability, traceability, safety, and user trust.
- Use varied test data, edge cases, failure scenarios, expected outputs, scoring criteria, and automated checks where practical.
- Validate robustness, accuracy, and reliability across key use-case conditions and user journeys.

DEMO READINESS

- Prepare realistic demo flows using quality input data, edge cases, failure paths, and expected outcomes.
- Show how AI, GenAI, or Agentic AI improves the solution versus a conventional approach in speed, scale, quality, reliability, or personalization.
- Clearly explain business value, adoption path, measurable outcomes, design choices, alternatives, trade-offs, and prototype-to-enterprise readiness.
- Walk through problem breakdown, data preparation, architecture, agent roles, prompts, tools, evaluation approach, code flow, and AI-assisted code generation.

Capital Markets Algorithmic Trading Strategy Advisor Agent

PROBLEM STATEMENT

Traders require rapid evaluation and adaptation of algorithmic trading strategies based on market conditions, historical performance, and risk profiles. Manual strategy tuning is complex and time-consuming, limiting responsiveness to market changes. Existing tools lack integrated AI capabilities for multi-agent collaboration in backtesting, risk assessment, and strategy recommendation. An intelligent agent that integrates market data, trading APIs, and historical performance can support traders in optimizing algorithmic strategies effectively.

Data Considerations:

Market data streams (tick, OHLCV) from financial APIs; historical trade and strategy performance records; trading platform APIs for order execution; moderate to high data volume; preprocessing includes data normalization and feature extraction; compliance with market data licensing; synthetic strategy datasets for development.

Consider using synthetic or anonymized data where appropriate. Ensure data quality and relevance to the problem context.

SOLUTION EXPECTATIONS

A multi-agent AI system with agents for data ingestion, backtesting simulations, risk scoring, and strategy recommendation. Features include a trader dashboard with performance charts, risk metrics, and suggested strategy adjustments. Integrations with trading APIs enable simulated or live strategy execution. Success metrics include recommendation accuracy, backtest performance improvements, and user feedback. Demonstrations showcase strategy evaluation workflows, multi-agent orchestration, and robust testing with documentation.

Feel free to explore creative solutions that align with the problem context and objectives.

AI Friday Season 2 – Regional Finale

Use the following checklist as applicable while solving the selected problem statement.

USER EXPERIENCE AND INTERFACE

- Design an intuitive interface aligned to the target workflow, user journey, context, tone, and accessibility needs.
- Use conversation, visualization, or a hybrid experience where it improves clarity, personalization, and speed of action.

DATA ARCHITECTURE AND PROCESSING

- Use relevant, high-quality, anonymized or synthetic data with clear preprocessing, retrieval, aggregation, and correlation methods.
- Choose suitable storage for structured and unstructured data, ensuring consistency across sources, generated datasets, and downstream outputs.

CORE AI SOLUTION DESIGN AND AGENTIC ARCHITECTURE

- Define the agentic design pattern, agent roles, collaboration model, handoffs, tool usage, retry, reflection, and escalation logic.
- Balance autonomy with human approval checkpoints, intervention points, and clear boundaries for high-risk or uncertain decisions.
- Ground outputs in trusted sources, cite evidence where useful, separate facts from assumptions, and explain reasoning, confidence, limits, and trade-offs.
- Build responsible AI guardrails for privacy, bias, explainability, prompt injection, unsafe recommendations, unauthorized tool execution, and regulated decisions.

TECHNICAL IMPLEMENTATION

- Use a modular, scalable architecture that clearly separates AI layers, enterprise systems, knowledge stores, APIs, tools, and external feeds.
- Define context engineering across memory, caching, retrieval, prompts, context windows, token usage, latency, cost, and privacy.
- Maintain configurability across models, prompts, tools, APIs, vector stores, and deployment environments to avoid lock-in.
- Cover security, identity, permissions, secrets, data retention, audit logging, access governance, and misuse monitoring.

TESTING AND QUALITY ASSURANCE

- Evaluate beyond demo success: accuracy, groundedness, hallucination control, latency, cost, reliability, traceability, safety, and user trust.
- Use varied test data, edge cases, failure scenarios, expected outputs, scoring criteria, and automated checks where practical.
- Validate robustness, accuracy, and reliability across key use-case conditions and user journeys.

DEMO READINESS

- Prepare realistic demo flows using quality input data, edge cases, failure paths, and expected outcomes.
- Show how AI, GenAI, or Agentic AI improves the solution versus a conventional approach in speed, scale, quality, reliability, or personalization.
- Clearly explain business value, adoption path, measurable outcomes, design choices, alternatives, trade-offs, and prototype-to-enterprise readiness.
- Walk through problem breakdown, data preparation, architecture, agent roles, prompts, tools, evaluation approach, code flow, and AI-assisted code generation.

AI-Powered Banking Customer Query Resolution and Fraud Alert System

PROBLEM STATEMENT

Banking customer service departments handle millions of inquiries daily, ranging from account information to transaction disputes and fraud alerts. Customers often face long wait times and inconsistent responses due to fragmented systems and manual case routing. Fraud detection teams struggle to prioritize alerts efficiently, leading to delayed responses that increase financial losses and customer dissatisfaction. Current chatbot solutions lack the ability to intelligently route complex queries or integrate real-time fraud data from multiple sources, limiting proactive risk mitigation. The scale of daily transactions and customer interactions amplifies the operational burden and compliance risks. An integrated AI solution that understands customer intent, automates fraud alert prioritization, and provides seamless multi-channel support is critical. This would reduce operational costs, enhance customer trust, and strengthen regulatory compliance by delivering timely, accurate, and personalized assistance.

Data Considerations:

Transactional data in JSON/CSV formats from internal banking systems; customer profiles from CRM APIs; real-time fraud alert feeds via REST APIs; historical customer queries and chat logs for conversation modeling; compliance guidelines datasets; data volume expected in the range of hundreds of thousands of records; preprocessing includes anonymization and normalization; privacy compliance with GDPR and PCI DSS must be ensured; synthetic data generation tools can simulate customer queries and fraud scenarios for testing.

Consider using synthetic or anonymized data where appropriate. Ensure data quality and relevance to the problem context.

SOLUTION EXPECTATIONS

A web-based AI assistant featuring multi-agent conversational modules that categorize and route customer queries intelligently, integrated with real-time fraud detection APIs to prioritize alerts. Key deliverables include an advanced UI with a supervisor dashboard showing alert statuses and resolution metrics, authentication for secure access, and multi-source data aggregation. Success metrics measured by query resolution accuracy, fraud response time reduction, and system uptime above 99%. Demonstrations will showcase real-time interaction flows, alert prioritization, and multi-agent collaboration resolving complex customer issues efficiently.

Feel free to explore creative solutions that align with the problem context and objectives.

Use the following checklist as applicable while solving the selected problem statement.

USER EXPERIENCE AND INTERFACE

- Design an intuitive interface aligned to the target workflow, user journey, context, tone, and accessibility needs.
- Use conversation, visualization, or a hybrid experience where it improves clarity, personalization, and speed of action.

DATA ARCHITECTURE AND PROCESSING

- Use relevant, high-quality, anonymized or synthetic data with clear preprocessing, retrieval, aggregation, and correlation methods.
- Choose suitable storage for structured and unstructured data, ensuring consistency across sources, generated datasets, and downstream outputs.

CORE AI SOLUTION DESIGN AND AGENTIC ARCHITECTURE

- Define the agentic design pattern, agent roles, collaboration model, handoffs, tool usage, retry, reflection, and escalation logic.
- Balance autonomy with human approval checkpoints, intervention points, and clear boundaries for high-risk or uncertain decisions.
- Ground outputs in trusted sources, cite evidence where useful, separate facts from assumptions, and explain reasoning, confidence, limits, and trade-offs.
- Build responsible AI guardrails for privacy, bias, explainability, prompt injection, unsafe recommendations, unauthorized tool execution, and regulated decisions.

TECHNICAL IMPLEMENTATION

- Use a modular, scalable architecture that clearly separates AI layers, enterprise systems, knowledge stores, APIs, tools, and external feeds.
- Define context engineering across memory, caching, retrieval, prompts, context windows, token usage, latency, cost, and privacy.
- Maintain configurability across models, prompts, tools, APIs, vector stores, and deployment environments to avoid lock-in.
- Cover security, identity, permissions, secrets, data retention, audit logging, access governance, and misuse monitoring.

TESTING AND QUALITY ASSURANCE

- Evaluate beyond demo success: accuracy, groundedness, hallucination control, latency, cost, reliability, traceability, safety, and user trust.
- Use varied test data, edge cases, failure scenarios, expected outputs, scoring criteria, and automated checks where practical.
- Validate robustness, accuracy, and reliability across key use-case conditions and user journeys.

DEMO READINESS

- Prepare realistic demo flows using quality input data, edge cases, failure paths, and expected outcomes.
- Show how AI, GenAI, or Agentic AI improves the solution versus a conventional approach in speed, scale, quality, reliability, or personalization.
- Clearly explain business value, adoption path, measurable outcomes, design choices, alternatives, trade-offs, and prototype-to-enterprise readiness.
- Walk through problem breakdown, data preparation, architecture, agent roles, prompts, tools, evaluation approach, code flow, and AI-assisted code generation.

Retail Customer Behavior Analysis and Personalized Promotion Agent

PROBLEM STATEMENT

Retailers struggle to analyze customer behavior across online and offline channels to deliver personalized promotions that drive sales. Fragmented data sources and lack of real-time insights hinder timely, relevant marketing offers. Existing solutions often deliver generic promotions, resulting in low engagement and ROI. An AI multi-agent system that integrates sales data, customer profiles, and external market trends can enable dynamic, personalized promotion strategies, increasing customer loyalty and revenue.

Data Considerations:

Sales transactions from POS and e-commerce systems; customer profiles from CRM; external market trend data; moderate data volume updated daily; preprocessing includes data cleansing, matching, and feature extraction; compliance with customer data privacy regulations; synthetic datasets with varied customer behaviors for development.

Consider using synthetic or anonymized data where appropriate. Ensure data quality and relevance to the problem context.

SOLUTION EXPECTATIONS

A multi-agent AI system featuring data ingestion agents, behavior analysis modules using pattern recognition, and promotion recommendation engines. Features include a marketing dashboard with customer segmentation, promotion scheduling, and performance analytics. Integrations with sales and CRM APIs enable data synchronization and campaign execution. Success metrics include promotion response rate, sales uplift, and campaign ROI. Demonstrations include customer behavior analysis workflows, multi-agent coordination, and real-time promotion adjustments with documentation and test coverage.

Feel free to explore creative solutions that align with the problem context and objectives.

Use the following checklist as applicable while solving the selected problem statement.

USER EXPERIENCE AND INTERFACE

- Design an intuitive interface aligned to the target workflow, user journey, context, tone, and accessibility needs.
- Use conversation, visualization, or a hybrid experience where it improves clarity, personalization, and speed of action.

DATA ARCHITECTURE AND PROCESSING

- Use relevant, high-quality, anonymized or synthetic data with clear preprocessing, retrieval, aggregation, and correlation methods.
- Choose suitable storage for structured and unstructured data, ensuring consistency across sources, generated datasets, and downstream outputs.

CORE AI SOLUTION DESIGN AND AGENTIC ARCHITECTURE

- Define the agentic design pattern, agent roles, collaboration model, handoffs, tool usage, retry, reflection, and escalation logic.
- Balance autonomy with human approval checkpoints, intervention points, and clear boundaries for high-risk or uncertain decisions.
- Ground outputs in trusted sources, cite evidence where useful, separate facts from assumptions, and explain reasoning, confidence, limits, and trade-offs.
- Build responsible AI guardrails for privacy, bias, explainability, prompt injection, unsafe recommendations, unauthorized tool execution, and regulated decisions.

TECHNICAL IMPLEMENTATION

- Use a modular, scalable architecture that clearly separates AI layers, enterprise systems, knowledge stores, APIs, tools, and external feeds.
- Define context engineering across memory, caching, retrieval, prompts, context windows, token usage, latency, cost, and privacy.
- Maintain configurability across models, prompts, tools, APIs, vector stores, and deployment environments to avoid lock-in.
- Cover security, identity, permissions, secrets, data retention, audit logging, access governance, and misuse monitoring.

TESTING AND QUALITY ASSURANCE

- Evaluate beyond demo success: accuracy, groundedness, hallucination control, latency, cost, reliability, traceability, safety, and user trust.
- Use varied test data, edge cases, failure scenarios, expected outputs, scoring criteria, and automated checks where practical.
- Validate robustness, accuracy, and reliability across key use-case conditions and user journeys.

DEMO READINESS

- Prepare realistic demo flows using quality input data, edge cases, failure paths, and expected outcomes.
- Show how AI, GenAI, or Agentic AI improves the solution versus a conventional approach in speed, scale, quality, reliability, or personalization.
- Clearly explain business value, adoption path, measurable outcomes, design choices, alternatives, trade-offs, and prototype-to-enterprise readiness.
- Walk through problem breakdown, data preparation, architecture, agent roles, prompts, tools, evaluation approach, code flow, and AI-assisted code generation.

Application Maintenance Intelligent Ticket Triage Agent

PROBLEM STATEMENT

IT support teams managing large-scale applications experience high volumes of maintenance tickets with diverse issues and priorities. Manual triage is time-consuming and inconsistent, leading to delayed resolution and reduced user satisfaction. Existing ticketing systems lack intelligent routing and prioritization based on issue severity, historical data, and resource availability. An AI-powered multi-agent system is needed to analyze incoming tickets, classify and prioritize them, and route them to the appropriate teams, improving resolution times and operational efficiency.

Data Considerations:

Historical and incoming ticket data in structured and unstructured formats (text, JSON); metadata including timestamps, issue categories, and resolution times; integration with ticketing platforms via APIs; moderate data volume with daily ticket influx; preprocessing includes text normalization and feature extraction; privacy compliance for sensitive customer data; synthetic ticket datasets with varied severity for development.

Consider using synthetic or anonymized data where appropriate. Ensure data quality and relevance to the problem context.

SOLUTION EXPECTATIONS

A multi-agent system with agents for ticket ingestion, NLP-based classification and prioritization, and routing to relevant teams or individuals. Features include an advanced UI for support managers to view ticket queues, priority scores, and routing decisions with override capabilities. Integrations with ticketing system APIs provide two-way synchronization. Success metrics include classification accuracy, average resolution time reduction, and routing precision. Demonstrations cover live ticket triage scenarios, multi-agent coordination, and test automation with full documentation.

Feel free to explore creative solutions that align with the problem context and objectives.

Use the following checklist as applicable while solving the selected problem statement.

USER EXPERIENCE AND INTERFACE

- Design an intuitive interface aligned to the target workflow, user journey, context, tone, and accessibility needs.
- Use conversation, visualization, or a hybrid experience where it improves clarity, personalization, and speed of action.

DATA ARCHITECTURE AND PROCESSING

- Use relevant, high-quality, anonymized or synthetic data with clear preprocessing, retrieval, aggregation, and correlation methods.
- Choose suitable storage for structured and unstructured data, ensuring consistency across sources, generated datasets, and downstream outputs.

CORE AI SOLUTION DESIGN AND AGENTIC ARCHITECTURE

- Define the agentic design pattern, agent roles, collaboration model, handoffs, tool usage, retry, reflection, and escalation logic.
- Balance autonomy with human approval checkpoints, intervention points, and clear boundaries for high-risk or uncertain decisions.
- Ground outputs in trusted sources, cite evidence where useful, separate facts from assumptions, and explain reasoning, confidence, limits, and trade-offs.
- Build responsible AI guardrails for privacy, bias, explainability, prompt injection, unsafe recommendations, unauthorized tool execution, and regulated decisions.

TECHNICAL IMPLEMENTATION

- Use a modular, scalable architecture that clearly separates AI layers, enterprise systems, knowledge stores, APIs, tools, and external feeds.
- Define context engineering across memory, caching, retrieval, prompts, context windows, token usage, latency, cost, and privacy.
- Maintain configurability across models, prompts, tools, APIs, vector stores, and deployment environments to avoid lock-in.
- Cover security, identity, permissions, secrets, data retention, audit logging, access governance, and misuse monitoring.

TESTING AND QUALITY ASSURANCE

- Evaluate beyond demo success: accuracy, groundedness, hallucination control, latency, cost, reliability, traceability, safety, and user trust.
- Use varied test data, edge cases, failure scenarios, expected outputs, scoring criteria, and automated checks where practical.
- Validate robustness, accuracy, and reliability across key use-case conditions and user journeys.

DEMO READINESS

- Prepare realistic demo flows using quality input data, edge cases, failure paths, and expected outcomes.
- Show how AI, GenAI, or Agentic AI improves the solution versus a conventional approach in speed, scale, quality, reliability, or personalization.
- Clearly explain business value, adoption path, measurable outcomes, design choices, alternatives, trade-offs, and prototype-to-enterprise readiness.
- Walk through problem breakdown, data preparation, architecture, agent roles, prompts, tools, evaluation approach, code flow, and AI-assisted code generation.